

14. Tabelle Hash



Lezione 29 - @November 28, 2025

Tabelle Hash

idea generale

- **dizionario** memorizzato in un array (tabella)
- la **posizione** di un elemento viene ottenuta applicando una *funzione* alla chiave

Funzione hash

U = universo delle chiavi

$\{0, \dots, m - 1\}$ spazio degli indici

Funzione Hash:

$$h : U \rightarrow \{0, \dots, m - 1\}$$

trasformazione di chiavi in indici (posizioni)

Tabelle ad accesso diretto

- chiavi univoche intere nel range $0, \dots, m - 1$ ($U = \{0, \dots, m - 1\}$)

Array di m posizioni: indice $\leftrightarrow$ chiave

$h :=$ funzione identità



ricerca, inserimento, cancellazione costano tempo $O(1)$

Fattore di carico α

$$\alpha = \frac{n}{m}$$

Quando la tabella è piena $\alpha = 1$

Quando la tabella è vuota $\alpha = 0$

$n \ll m \rightarrow$ spreco di spazio.

Funzioni hash perfette

$h : U \rightarrow \{0, \dots, m - 1\}$ è **perfetta**

se $u \neq v \Rightarrow h(u) \neq h(v) \equiv$ iniettiva

Funzioni hash

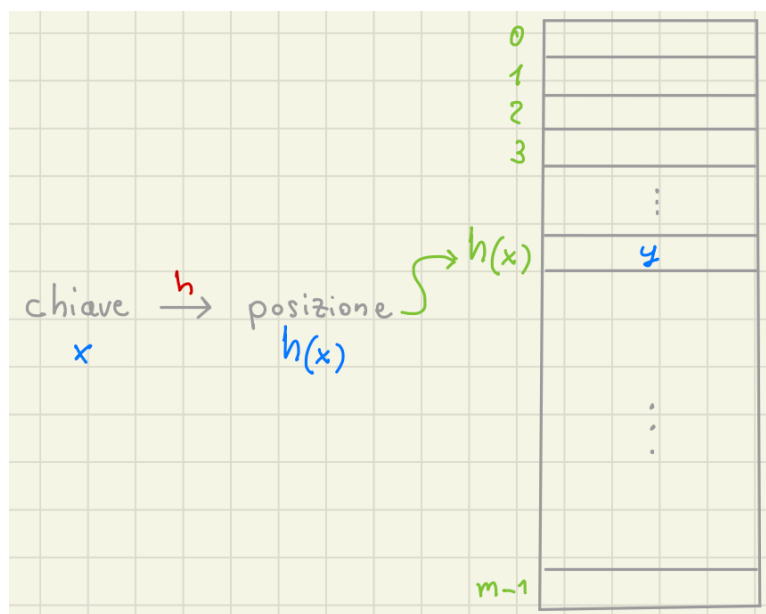
nella pratica, salvo casi particolari:

- il numero di chiavi possibili $\#U$ è molto più grande del numero di chiavi attese
- la dimensione della tabella m viene scelta paragonabile al numero di chiavi attese

Cognomi $\leftrightarrow$ stringe se $U =$ stringhe di 10 lettere

$$\#U = 26^{10} > 10^{14} \approx 141\,167 \text{ miliardi}$$

Collisioni



Una **collisione** si verifica quando dovendo inserire/cercare una chiave x , la posizione $h(x)$ è occupata da un'altra chiave y .

"convivere" con le collisioni

- fare in modo che avvengano raramente
- h deve sparpagliare le chiave nella tabella in modo che le collisioni siano poco probabili

$\Rightarrow$ scelta della funzione di hash

- uniforme
- veloce da calcolare
- avere una strategia che gestisca le collisioni (nel caso accadano)

Funzioni Hash

- $h : U \rightarrow \{0, \dots, m-1\}$ funzione hash
- Per $x \in U$:
 $\mathcal{P}(x)$: probabilità che, scegliendo a caso una chiave da U si scelga x
- Per $i = 0, \dots, m-1$
 $\mathcal{Q}(i) = \sum_{x:h(x)=i} \mathcal{P}(x)$ probabilità che una chiave scelta a caso da U abbia valore hash i



La funzione hash h è **uniforme** se $\mathcal{Q}(i)$ è la stessa per ogni i , cioè $\mathcal{Q}(i) = \frac{1}{m}$

Esempi di funzione hash

metodo della divisione

- 1. trasformo la chiave in un intero
- 2. divido per la dimensione della tabella e considero il resto

$$h(x) = x \text{MOD} m$$

x è in binario

m è solitamente potenza di 2.

- In generale la dimensione della tabella è una potenza di 2.
- Il valore della funzione hash deve avere dipendenze da tutta la chiave e non solo da una parte di essa

HASH $\equiv$ sminuzzare/triturare

metodo del ripiegamento

- m potenza di 2, $m = 2^k$
- x chiave in binario
- suddivisa in blocchi: $x_1, x_2, \dots, x_l$ di k bit ciascuno
- $f(x) = x_1 \oplus x_2 \oplus \dots \oplus x_l$

esempio:

$m = 2^{10}$

$x = GATTO = 0100011101|0000010101|0100010101|0001001111$ blocchi da 10 bit ciascuno

$$\oplus \begin{cases} x_1 = 0100011101 \\ x_2 = 0000010101 \\ x_3 = 0000010101 \\ x_4 = 0001001111 \end{cases}$$

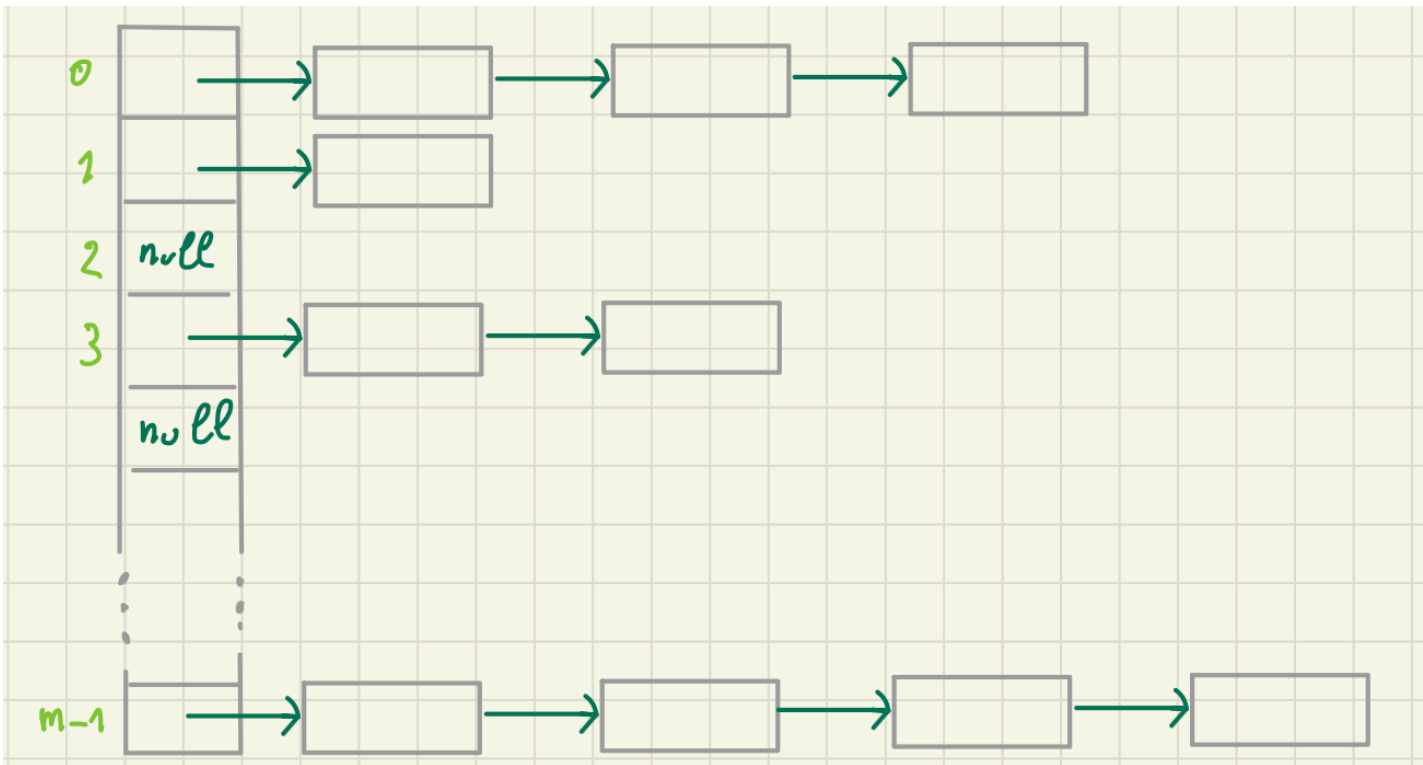
$$f(x) = 0101010010 = 338$$

- facile da calcolare
- buona uniformità (xor ottimizza le alternative)

Gestione delle collisioni

Tecniche di gestione esterne/interne

Lista di collisione (concatenamento esterno)



La posizione i contiene ogni record la cui chiave x ha valore hash i , cioè $h(x) = i$

Operazioni

- `inserimento(elemento e, chiave k)`
 $O(1)$ se l'elemento viene inserito all'inizio.
- `ricerca (chiave k) → elemento`
- `cancellazione (chiave k)`

per ricerca e cancellazione: #passi dipende dalla lunghezza della lista

- caso peggiore $O(n)$
- in media lunghezza lista $\frac{n}{m} = \alpha \rightarrow T_{avg}(n) = O\left(1 + \frac{n}{m}\right)$

Agglomerazione

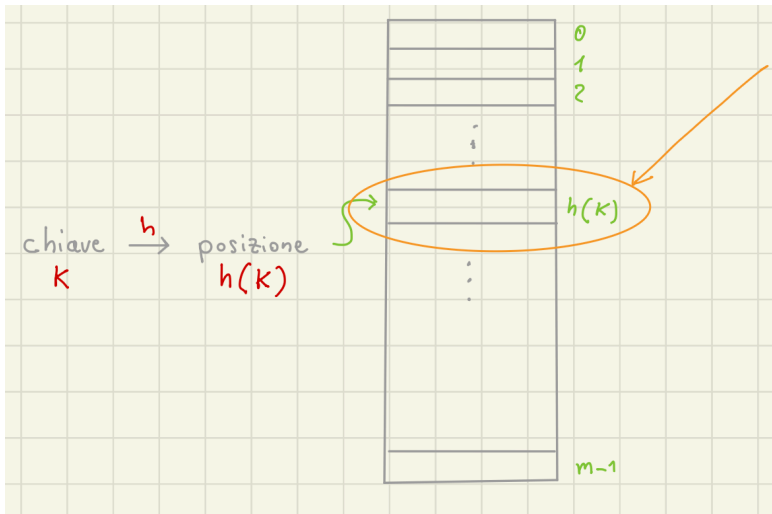


presente in zone in cui si concentrano molti dati

Se la funzione hash non sparpaglia bene si potrebbero formare alcune liste molto lunghe

indirizzamento aperto

inserimento di elemento di chiave k



Se $h(k)$ è già occupata si cerca un'altra posizione libera utilizzando una strategia predefinita

scansione lineare: scelgo la prima posizione libera successiva $\lfloor \text{mod } m \rfloor$

agglomerazione primaria

$$c(k, i) = \left(h(k) + i \right) \text{MOD } m$$

funzione ausiliaria $c(k, i)$, k chiave e i intero ≥ 0 ordine nella scansione

La funzione ausiliaria deve verificare:

- $c(k, 0) = h(k)$
- $\{c(k, 0), c(k, 1), \dots, c(k, m - 1)\} = \{0, \dots, m - 1\}$

scansione quadratica

$$c(k, i) = \lfloor h(k) + c_1 i + c_2 i^2 \rfloor \text{MOD } m \text{ con } c_1, c_2 \text{ opportuni (es } c_1 = c_2 = 0, 5)$$

nota

se $h(k_1) = h(k_2)$ la sequenza di celle scansionata per k_1 e k_2 è la stessa

Possibilità di **agglomerazione secondaria**

Hashing doppio

$$c(k,i) = \left(h(k) + ih'(k)\right) \text{MOD } m$$

dove $h' : U \rightarrow \{0, \dots, m - 1\}$ è una seconda funzione di hash.

situazione ideale

$$h(k_1) = h(k_2) \Rightarrow h'(k_1) \neq h'(k_2)$$

possibilità di agglomerazione secondaria.

Implementazione pseudo-codice

`v[0...m-1]` tabella hash

```

FUNZIONE Inserisci(elemento e, chiave k)
  i <- 0
  WHILE i < m AND v[c(k,i)] è occupata DO
    i <- i + 1
  IF i < m THEN
    v[c(k,i)] <- (c,k)      // inserisci elemento
  ELSE errore              // errore!

```

```

FUNZIONE Ricerca(chiave k) -> elemento
  WHILE i < m AND v[c(k,i)] è occupata AND v[c(k,i)].chiave != k DO
    i <- i + 1
  IF i = m OR v[c(k,i)] è libera DO
    RETURN null            // non c'è
  ELSE
    RETURN v[c(k,i)].elemento // trovato

```

La cancellazione avviene solo in maniera logica, ovvero l'elemento viene *flaggato* come eliminato ma permane nella tabella poiché una volta costruita gli elementi hanno quella posizione FISSA.



Lezione 31 - @December 3, 2025

Funzioni ausiliarie: esempi

scansione lineare

$$c(k,i) = \left(h(k) + i\right) \text{MOD } m$$

scansione quadratica

$$c(k,i) = \lfloor h(k) + c_1i + c_2i^2 \rfloor \text{MOD } m$$

hashing doppio

$h' : U \rightarrow \{0, \dots, m - 1\}$ è una seconda funzione di hash

$$c(k,i) = \left(h(k) + ih'(k)\right) \text{MOD } m$$

per la ricerca devo applicare la funzione di hash doppio fino a quando non trovo la cella vuota o la chiave che sto cercando.

Costi operazioni - gestione interna

i costi delle operazioni sono strettamente influenzati dal costo di scansione

nel caso peggiore $\Theta(n)$ passi per scandire tutta la tabella.

se la funzione hash “sparpaglia” bene il caso medio è molto inferiore.

n° di passi	scansione lineare	scansione quadratica hashing doppio	$n = \# \text{ chiavi presenti}$ $m = \text{dim tabella}$
chiave trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)}$	$-\frac{1}{\alpha} \lg_e(1-\alpha)$	
chiave non trovata	$\frac{1}{2} + \frac{1}{2(1-\alpha)^2}$	$\frac{1}{1-\alpha}$	$\alpha = \frac{n}{m}$ FATTORE DI CARICO

$$\frac{1}{1-\alpha} \rightarrow \begin{cases} \alpha = \frac{1}{2} \Rightarrow \frac{1}{1-\alpha} = 2 \\ \alpha \rightarrow 1 \Rightarrow \frac{1}{1-\alpha} = 4 \\ \alpha = \frac{3}{4} \Rightarrow \frac{1}{1-\alpha} \rightarrow \infty \end{cases}$$

il numero di passi di calcolo dipende dal fattore di carico α , cioè da quanto è piena la tabella

bisogna tenere sotto controllo il fattore di carico $< \frac{1}{2}$

Se si supera questo limite $\rightarrow$ re-hashing (nuova tabella)

Rehashing

quando una tabella viene riempita oltre la soglia stabilita ($\frac{1}{2}$) viene creata una tabella più grande (di solito il doppio) in cui vengono trasferiti tutti gli elementi.

Problemi

- funzione di hash per la nuova tabella
- costo in termini di tempo

Nuova funzione hash

$f(k)$: funzione che restituisce un intero in un range molto grande (più grande della tabella)

$h_m(k) = f(k) \text{ MOD } m$ funzione parametrica a dimensione della tabella, se f è uniforme $\Rightarrow$ anche h_m è uniforme.



le mappe in Go sono implementazioni di tabelle hash

quanto costa il re-hashing?

- tabella inizialmente vuota
- sequenza di operazioni tra cui N inserimenti
- Re-hashing ogni volta che α supera $\frac{1}{2}$, raddoppiando la dimensione della tabella

Si dimostra che:

- il numero complessivo di inserimenti dovuto alle operazioni di re-hashing è $O(N)$
- se ciascun inserimento costa $O(1)$ il costo ammortizzato dell’operazione di inserimento rimane $O(1)$